

Manual — Skill Engineering

What this is

Why it exists

How it works, end to end

The one hard boundary: `claude_code` only

What enforcement does and does not mean

Recommended pairings — not enabled by default

Gotchas worth knowing before you hit them

A note on scope

Version history

A candidate first chapter of a broader SSSF user manual. Scope and structure — standalone document vs. a chapter alongside the adoption playbook — is an open decision; see “A note on scope” at the end.

What this is

SSSF decides *where and when* an agent runs — phases, gates, the envelope. It says almost nothing about *how rigorously* the agent works once it is running. A body of well-tested engineering protocols already exists as `SKILL.md` files — `tdd`, `codebase-design`, `diagnosing-bugs`, a grilling protocol — the missing “how.” Skill engineering is the mechanism that gets one of those protocols into a headless SSSF agent’s hands.

Concretely: you vendor a skill file into your repo, name it on an agent in `sssf.config.yaml`, and at call time SSSF composes that agent’s system prompt with the skill’s text appended. The agent runs under the same discipline it would follow interactively — but reproducibly, committed, and without opening a door SSSF deliberately keeps closed.

Why it exists

SSSF runs agents with `--setting-sources ''`, which makes every run hermetic and — measured — cuts prompt overhead from 22,478 to 17,932 tokens per turn. That same flag hides every skill installed on the operator’s machine. An agent that would follow `tdd` in an interactive session has no idea the skill exists once it’s running headless.

The naive fix — dropping `--setting-sources ''` for agents that need a skill — was considered and rejected. Re-enabling native skill discovery also re-admits the operator’s hooks, plugins, and MCP servers, which makes a run’s behavior depend on whichever machine happened to run it. That is the opposite of what a factory is for.

Skill engineering routes the protocol in as **text**, not as a discoverable tool. The skill’s body is vendored into the repo, reviewed in a diff, and appended to the agent’s system prompt at call time — reproducible on any machine that has the same commit checked out, because the discipline is a file, not an environment.

How it works, end to end

1. **Vendor a skill.** `uv run <skill>/scripts/vendor_skill.py ~/.claude/skills/tdd/SKILL.md` copies the file into `adws/adw_data/skill_engineering/tdd.md`, stamped with a provenance header — source path, date, a content hash. Every real Pocock skill file is literally named `SKILL.md`; the destination name is taken from the *parent directory*, not the filename, so vendoring several skills never collides them into one file.

2. **Attach it to an agent.**

```
agents:
  - name: builder
    skill_engineering:
      - adws/adw_data/skill_engineering/tdd.md
```

Or house-wide, inherited by every agent that doesn't set its own:

```
defaults:
  skill_engineering:
    - adws/adw_data/skill_engineering/tdd.md
```

A per-agent list **replaces** the default; it never appends. An agent that omits the key inherits the default; an agent that sets `[]` explicitly gets no skills even with a default in place.

3. **The agent runs.** At call time, SSSF composes the agent's own `system.md` first, then each named skill's text, in the order listed — never sorted, because the agent's identity and output contract must outrank any borrowed protocol, and a stable order is what keeps the prompt cache working. The composed text is what actually reaches the model, and it is persisted to the session directory next to the agent's other prompts, so what the agent was told is recoverable after the fact.
4. **You see the cost before you pay much of it.** The console reports the skill names and an estimated token cost right after the agent starts — a `chars/4` heuristic, always labelled “est.”, never presented as a billed count. A soft budget (`skill_token_budget`, also roster-wide or per-agent) warns when a composed prompt gets big. It never fails the run — you may have decided the discipline is worth the money.

5. **You can audit it later.** `just skills` lists every vendored file and the agent names that actually receive it, separating agents that named a skill but whose `coding_agent` means it never applies.

The one hard boundary: `claude_code` only

Skill engineering takes effect **only** under `coding_agent: claude_code`. Skills ride in `--system-prompt`, a delivery mechanism specific to how SSSF drives Claude Code headlessly; `pi` and `agy` have no equivalent path for it.

Naming `skill_engineering` on a `pi` or `agy` agent is not a config error — a roster might genuinely be mid-migration between coding agents — but it does nothing, and SSSF says so: `agents.validate()` prints a warning naming the agent, before anything spawns, and `just skills` marks that agent as `[ignored by: ...]` rather than counting it as a user.

This boundary was tightened during the feature's own build: an early version's warning correctly *said* the field was ignored under `pi / agy` while the composition code actually injected and billed it anyway on every one of those agents. The fix — one function, `skill_engineering_applies(agent)`, called from every place that needs to know — is now the single source of truth every other check (the console report, the trace, the audit) derives from, specifically so the claim and the behavior can't diverge silently again.

What enforcement does and does not mean

Outcome gates only. No gate ever attempts to prove a protocol was followed — no inspecting intermediate commits for a test written first, no requiring the envelope to cite evidence of the discipline. `tdd.md` shapes *how* the builder works; the suite, the linter, and the typechecker judge *what came out*. A gate that claims to verify process but can be satisfied by a well-worded envelope is worse than no gate — it is the placeholder problem in a new costume.

This means attaching a skill is a claim about behavior, not a guarantee enforced by the factory. If you want to know whether it actually changed what an agent produced, the only honest way is the one this manual doesn't shortcut: run the same request twice, once with the skill attached and once without, from an identical baseline, and read both diffs and both traces yourself.

Recommended pairings — not enabled by default

The starter roster ships with no `skill_engineering` set anywhere. Adding unrequested per-turn cost to every fresh install would be wrong.

Agent	Skill	Why
<code>builder</code>	<code>tdd</code>	A feature arrives with tests written to fail first.
<code>reviewer</code>	<code>codebase-design</code>	Judges module depth and interface quality, not just correctness.
<code>planner</code>	a grilling protocol	Resolves ambiguity before the expensive build phase — but verify it first: the skill assumes a human is present to answer questions, and a planner with nobody to grill in a headless run could stall or invent answers.
fix-loop builder	<code>diagnosing-bugs</code>	A red gate produces a diagnosis, not a guess.

Opt in per roster, one pairing at a time, and read the diff after the first run either way.

Gotchas worth knowing before you hit them

- **Vendoring never overwrites a hand-authored file.** If `adws/adw_data/skill_engineering/tdd.md` already exists and has no provenance header, the vendoring tool refuses and touches nothing — that file was written by a person, and the tool does not get to decide it was actually meant to be a vendoring target.
- **Re-vendoring unchanged content is a no-op**, including the file's date stamp, so a re-vendor loop or a CI check never produces a spurious diff.
- **Nothing auto-updates.** `vendor_skill.py --check <file>` reports drift against the original source — moved, deleted, or changed — but never resolves it. Re-vendoring on purpose is how drift gets fixed.

- **Provenance headers are portable, not personal.** A vendored file's header stores its source relative to `$HOME` (as `~/.claude/skills/...`) when the source lives there, which is the common case — not as an absolute path that would bake one machine's username into a committed file and break drift-checking for anyone else who clones the repo.
- **The composed prompt is billed on every internal turn** of a phase — the first send, every JSON-retry, every gate-correction — not once per agent call. A 3,000-token skill on a nine-turn build phase is real money; `skill_token_budget` exists so that price is visible at the moment you're paying it.

A note on scope

This manual currently covers one feature. The adoption playbook (`docs/playbook-adopting-sssf.md`) covers running SSSF against real work — recon, wiring gates, the four jobs, reading what happened. Whether these belong as two separate, independently-versioned documents, or as chapters of one SSSF manual with the playbook as its practical/operational chapter and this as a features-reference chapter, is not yet decided. This file is written to work either way: as a standalone document today, or folded into a larger manual later with its own `##` heading demoted to `#` and its version history merged into the parent document's git history rather than tracked here.

Version history

Version	Date	Changes
1.0	2026-08-31	Initial manual. Covers the <code>skill_engineering</code> feature end to end, post the branch's adversarial-review correction rounds (the <code>claude_code</code> -only boundary is now enforced, not just claimed).